HCI Principles

IUM 2024/2025

people are complex

- individual differences: ≠ thinking styles, ≠ interaction strategies
- parallel processing: people may handle ≠ jobs at the same time - how can we use this?
- expert systems: it is convenient to view the mind as an expert system: LTM a body of rules, STM a blackboard, the number of goals is limited, access to rules in LTM increases their probability of future access...
- wider issues: motivation, stress, play, environment affect the user and therefore his attitudes

skill/rule/knowledge

3 levels of abstraction

1. **skill-based** performance runs without conscious attention or control, e.g. typing for a typist - it works only if the user's control model is an adequate representation of the interface
2. **rule-based** performance is the recollection of skilled behaviour patterns
3. **knowledge-based** behaviour is used for an interaction poorly rehearsed so that rules have not been developed - goals must be explicitly formulated in order to work out a plan

The system will favour the transition from knowledge-based to skill-based user behaviour

symbolic/atomic/continuous

three human modes ≡ the interface styles

- symbolic - textual commands with non-trivial syntax, example:

  delete *.txt (operation) (expression)

evaluating to a list of file names ending in .txt

but this will be executed only after the user will press

enter

to finally perform the *delete* operation.

Symbolic interfaces are often extensible

- atomic - special purpose keys, perceived and executed immediately, trivial syntax, either succeed completely or have no effect, quantified, no composition available (menu interfaces are atomic)

last mode

- continuous - using continuous input devices like a light pen, mouse, joystick with fast visual feedback for effective use - eye, finger, hand or arm movement even body
- selection-oriented, they support atomic input style, also used for supplying parameters to a symbolic interface
- A soft keyboard (an on-screen keyboard) allows users to input characters that might not be available on a physical keyboard, such as those from extended character sets (e.g., accented letters or special symbols).

interface styles, other differences

- each style is distinct, but can emulate the other two - a real interactive system will mix them all - incremental interaction is based on the atomic style - a symbolic style is supported using a timesharing system - like in banks - ideal for mainframes
- once a symbolic command is entered it may entail an arbitrary amount of number crunching without providing feedback to the user - only the final result (a receipt) -
- an atomic interface requires feedback to the user per stroke (about 10/second) needing more computing power
- a continuous interface requires a real-time response, the cursor must keep up with a pointing device, sometimes the user wants to move/rotate an object and more computing power is required like with a workstation or a dedicated personal computer

trade/offs

- error in a symbolic interaction results in losing the submission - it may be recovered depending on the user's LTM
- a system may provide naming as a way of expediting chunking - the user may remember the name in LTM more easily -
- error in an atomic interaction causes a single action to be lost, recovery depends on STM, if the error is not reported soon enough, it may be lost from STM
- errors in a continuous interaction arise continuously...and the user keeps correcting them, these systems are reversible since over-shoots and under-shoots often occur

if the feedback is fast (per-action) we have reactive systems which include the interactive ones

linguistic aspects

- lexical form of commands/form of output symbols
- syntactic combination rules of previous commands
- semantic what do these combinations mean

When an interface processes the input, the lexical level is associated to the way symbols are presented on the screen by the system (colours, windows, widgets, menus, icons, etc.) while the syntactic level is tied to the rules for generating such visual symbols, for displacing windows, for positioning them and managing screen estate, for rotating, enlarging, etc. all displayed symbols; finally the semantic level cannot be defined with a-priori rules but is strictly conditioned by the tasks that must be executed.

errors

- all these levels are interwoven...to find out the useful levels HT suggests to distinguish them on the basis of how they fail to work in a program, i.e. how long it takes to detect an error and what are its consequences
- lexical errors can be detected almost immediately - next syntactic errors may be located after checking the rules, finally, semantic errors can be found: the first error is easy to recover from (just submit the correct lexeme), the second one may have propagated backwards and forwards but the system just reports it, finally the last error (semantic one) may have drastic consequences since an action was performed which was not the intended one.

what is **good** design?

design is a synthetic science attempting to understand synthetic worlds where users live:

1. design should be **task-specific** - and the task should be clearly stated
2. design should have **predictable performance** - in general, it is impossible to know what you buy until you break the seal
3. design should be **iterative** - prototype as a starting point for a new design cycle
4. design has **more control than evaluation**

Is there a theory of interface design? if yes, should such theory provides: explanations - predictions - performance - ???

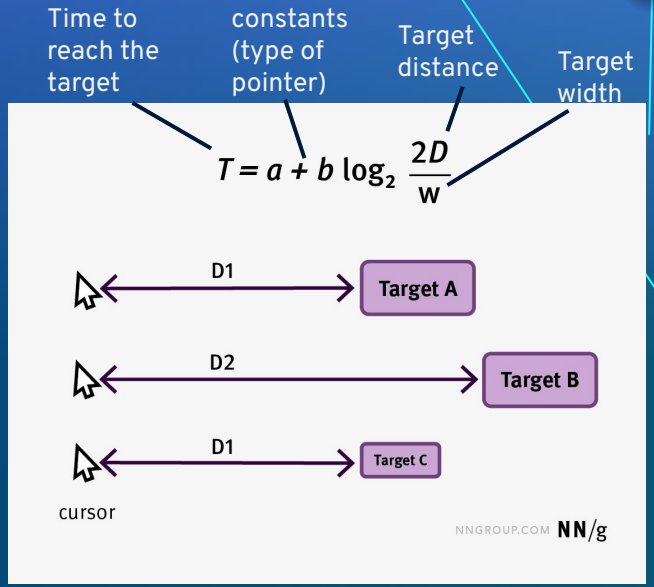
the problem is: how does a human and a machine communicate?

guidelines

- attitudinal guidelines are meant to raise consciousness identifying issues for which there is no general answer.
- a standard guideline is "present informative feedback" - but this is **too general** and also in some cases wrong, like when you are designing a data-safe input system and the user failed his password.
- input and output should be compatible but in some cases compatibility might be a wrong choice as when entering a date: 97/06/10 does it mean tenth of june or sixth of october? the system should respond showing "his" interpretation so that the user may check, and see if the date he entered had the correct format.

Fitt's law

- the **time** to move your hand (with a mouse) depends **logarithmically** on the ratio of the **target** size to the **distance** you have to move the hand to hit the target - i.e. it is not a linear relationship
- this law may help in deciding on sizes of more distant objects on the screen: they must be bigger
- guidelines are hints - personal attitudes, abilities, tastes are important for designing a good interactive system - the designer should know the user and let him know that he cares...



https://www.nngroup.com/articles/fitts-law/

vagueness

- users evolve: from naïve to experts, from naïve to informed, from children to grown-ups, from working in an office to working at home, etc.
- vague principles allow designers to adhere to them without objective evaluation
- but vagueness is dangerous...

Sorites paradox

- an interactive system with no commands is not easy to use
- adding a command to a difficult-to-use system does not make the system easy to use (adding a help command may make it easy to learn, not to use)
- in this way no number of commands makes a system easy to use - the fault is that 'easy to use' is not predicated on the number of commands/that easy to use is a vague concept which, incidentally, does not help in designing a system having that property

principle of least astonishment

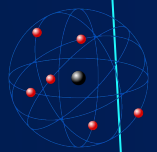
- problems arise at boundary conditions, the idea is to surprise the user as little as possible, particularly when entering new modalities/exceptions/difficult conditions for the user

- for whom is astonishment minimized? subjective criteria

- performance against uniformity - not everything may be undone...

new orientation

- some people can both establish and use the rules for working - interface design has one person designing another using the result of such design
- good interactive systems are born by inspiration - guided by formal considerations - partly by ad hoc accretion...generally a coherent system does NOT result which means difficulties to introduce the system to new users, particularly if they are casual or non-experts
- *Rutherford* said that a scientist does not really know what he is doing, unless he can explain his work to the laboratory cleaner...
- guidelines should be accessible to both the designer and the user



GUEPS: a way to model principles

Thimbleby introduced the concept of generative user engineering principles (GUEPS). These principles have several properties:

- They apply to a large class of different systems. That is they are generic.
- They can be given both an informal colloquial statement and also a formal statement.
- They can be used to constrain the design of a given system. That is they generate the design.

Modes

- role of hidden information - interactive system *modes* which are generally used to increase versatility of the system, yet they create increasing confusion
- mode: variable info in the computer system affecting the meaning of what the user sees and does - editing, scrolling, drawing, zooming, etc. within the same application - also called views
- some people consider input modes but it is better to view i/o parametrically

mode features

- typically the system gives no clue to the user about its present state or how it was reached (hidden info)
- nobody agrees on what a mode is: info channel in psychology/classes in logic/peaks in statistical distributions...
- mody systems can be used faster but ambiguity and complexity are the two main drawbacks

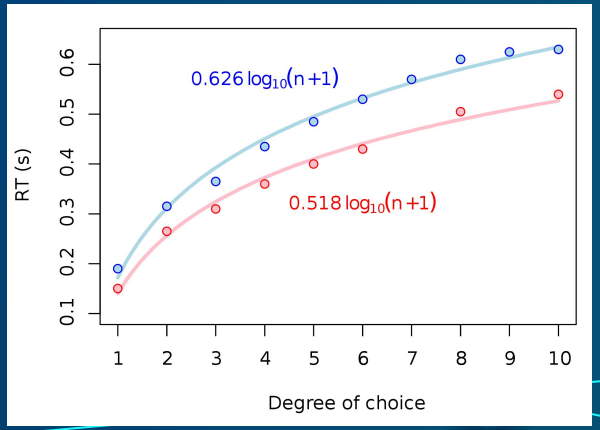
low mode or modeless?

- strictly speaking, a modeless system cannot work! it should be switched on - which is a ≠ mode from the off condition
- yet, modelessness is perceived as a quality - in fact low mode systems is a more adequate name: the user does not perceive the modes and works without effort
- a public-access system is truly modeless: a kiosk may be always on with a few buttons for fixed queries on local information

Modes: a negative issue

- if too many modes, the user loses track...difficult to remember in which mode he is - Hick's Law (1952) states that “the user's decision time is logarithmic with the number of choices known to him”...sometimes the user does not know such number nor the exact meaning of each choice

https://en.wikipedia.org/wiki/Hick's_law



spatial, temporal modes

- why modes? because we only have about 70 keys on the keyboard, a small screen and a small bandwidth for the i/p medium (fingers/keys) with a large bandwidth for the o/p medium (eyes/screen)
- slow transmission results when using few keys, fast transmission for a large number of them occupying a larger space (time vs space) but there is graceful degradation if time is chosen while abrupt degradation exists for a space choice
- the mode in the user head may be out of pace with the mode in the computer system...
- meaning of the mouse button depends on where the mouse is pointing: spatial mode instead of temporal mode

cumbersome modes...

- it is difficult to provide cues (visual, aural) to the user to remind him or tell him in which mode he is moving around

- temporal modes appear when pressing a button on the mouse has a meaning depending on the menu choice (transient spatial), if no button is pressed the system reverts to a basic mode so giving a robust feeling to the user

illustrating modes

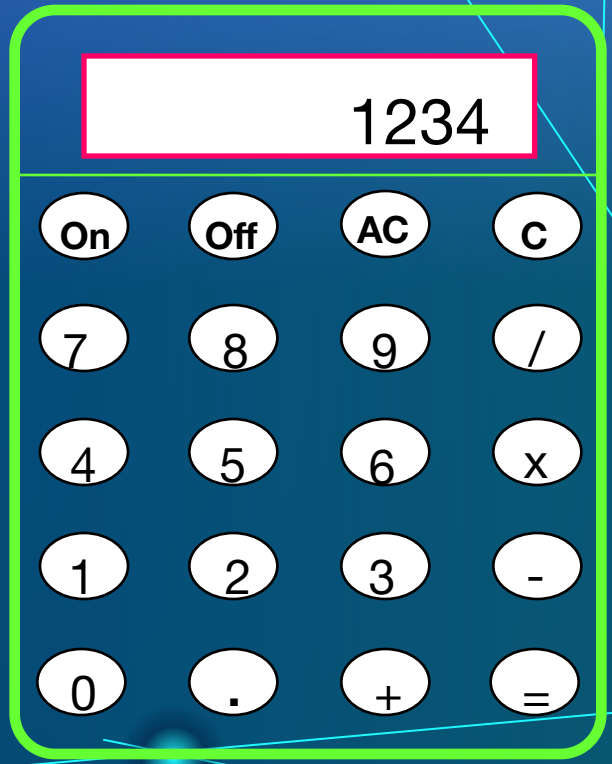
- a backspace character may be misleading: what does it mean if it shows on the screen? what happens to existing characters? etc.
- let us consider a 4-function calculator:
- the display is used for many different things at different times and there is no way for the user to tell which one except by remembering what he has done in the recent past...
- some significant info is never displayed and the user always has to remember...

the calculator example

- this calculator has 20 buttons, the On button has no use except to switch the calculator on until we want, and to switch it off by pressing the Off button
- if you want to use this calculator and nothing is displayed press On, a 0 will be displayed
- if the user hits a digit (except for 0) such value is added to the display appearing from the right - as if we multiply each time by 10 - in fact adding 0 and multiplying it by 10 it is still 0
- if the decimal point is hit (second button from the left in the bottom row) then numbers are divided by 10 until the display is full

trying to use the calculator

- what happens next (if display is full) depends on whether the user hits the decimal point: if no, the display gets jammed up, the C button must be pressed and a 0 reappears, if yes further digits are ignored and the display remains as it was
- so far it is difficult but understandable due to the knowledge users have of number representation and manipulation

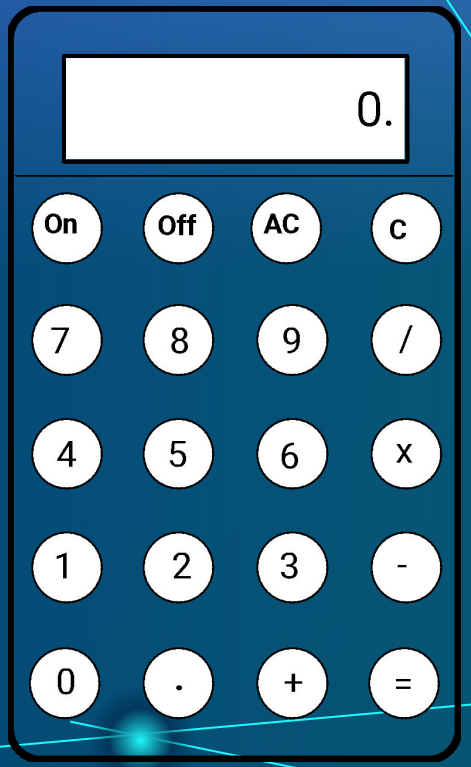


understanding the calculator

- if an arithmetic button is pressed, there is no effect but when another digit is pressed a new number appears - what is hidden (because we have only one display) is the fact we are working with 2 numbers:

1. number being entered by user
2. number representing the running calculation

- suppose we have a value (partial result) on the display, we press + and then another number = the total result (adding the new number to the previous value)



bi-modal calculator

1) hitting a digit: display changes to current number
2) hitting an operator: display changes to calculation

- since buttons generally give no feedback, they are small, may be pressed too lightly, etc. the user is never certain whether he has pressed the button strongly enough, so
- there is confusion about which value is displayed: the current number or the running calculation?
- this calculator has 2 modes but there is no way to tell!

how many modes?

- other calculators have even more modes: pressing an operator twice means that the running calculation may be copied to the constant register, if / is pressed twice it may mean a different operator (the inverse), special keys perform percentages, etc.

- there is no way to find out which mode the calculator is in, without affecting it (invasive check-out)

inertial modes

- the system is able to fill-in details by defaulting them, info provided by default is hidden - there are also user-supplied defaults

square width ? *1.2471 enter*

- ...

square width ? 1.2471? *enter*

- this is a mechanism of passive recall since the system will ask the user - by default - if the value he now wants is the same as the previous one (default value)

inertial dangers

- defaulting is not a good choice if the result is irreversible
- some menus require two actions:
- 1-choice and
- 2-confirmation, the last choice is inertial - the default choice is the last one the user selected
- the same happens when using many menus - one after another - going back one step to a previous menu will provide the same choice made in a previous session (this is the default value)
- some forms of modiness may turn useful and are called inertial yet sometimes the user is disoriented or cannot do what he wants - then we have modes

gueps for modes

input modes

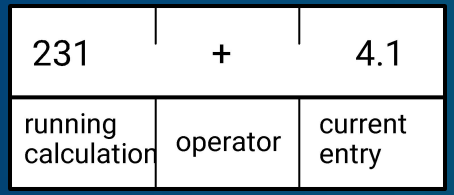
- the user submits *q* in one mode it may mean QUIT, in another “type *q* “ corresponds to another mode which may mean an ERROR; the corresponding guep indicates that a good interactive system is one which is so consistent and predictable that it may be used without looking at the screen
- if there is more than one mode, the user may switch from one to another by fixed mode-independent command sequences
- for instance, *escape* will take the user to a fixed state - external to the currently running application - even if that action is submitted a number of times...this is the idempotence property (*escape escape* = *escape*)

pre-emptive modes

- for outlawing pre-emptive modes you may have a guep saying: "you may do anything anywhere"
- for non pre-emption:
  - using a temporal feature: every user-command can be entered if prefixed by a cursor (! for instance)
  - using a spatial feature: with ≠ windows popping up one may change mode in another part of the screen, typically inside a new window

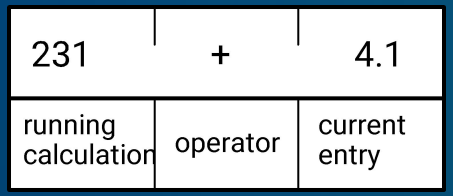
output modes

- the o/p mode is typically dependent on the history
- in the calculator case an imperative paradigm was used and 2 modes were present; if the system was declarative, no ≠ modes are necessary, the user may be guided only by what he can see (on the display) / showing what is relevant



a better calculator interface

- in this way, the user can do sums on what he can see alone - a good design would show all of the calculation that is relevant to the user
- nothing is hidden - every button has a well defined meaning which is shown on the display
- the only relevant entities are those that can be perceived - the "=" button is now redundant



WYSIWYG

What You See Is What You Get (wizzy-wig) = no hidden info

- previously, for a text processor/editor one had commands (.CENTRE) for centring a line, but the only way to see the effect was to print the line
- ww avoids the problems of programming "blindly" to format the text as wanted but limits the control of a sophisticated programmer
- a compromise is to have ww for simple things and free programming for advanced features
- ww pushes both designer and user to share conceptualization of the interactive system
- the good features of a ww system which make it easy to use are tedious for an expert to use

ww

- in ww you always see the end result, the user may concentrate on contents and then on form or viceversa - the system is underdetermined, you cannot see the history, i.e. the evolution of the formatting process, the process of improving a drawing (inside a text) is one of always editing the last version since the first one is lost

- some of these problems may be solved with undo and redo

domain independence?

- a good guep is to ignore the application when designing the interface
- in ww we consider a style for interactive text processing systems but could also hold for drawing or for any application which will print out what one is obtaining as a result on the screen

menu systems

- menu systems are WYSIWYG - based, all choices are given but if more than those shown are available then we do not have WYSIWYG any more!
- if typing a number which is higher than those indicating the different menu choices produces a result (unexpected) this - again - is not WYSIWYG style
- all choices should be visible on the screen when selecting the menu label on top of the different possible paths

interpretation of commands

- task structuring is a guide to modes
- commands of a non-modey system (single mode) change the system state
- they map the current state into the next state which may formally be written as

  I [c] ∈Σ → S meaning that the Interpretation of the command c changes the current state of the system to the next State

as an example, imagine we type

  i to a word processor, its meaning is

  I [i] ∈Σ → S changing the state of the word processor into the next state

for the calculator case

- suppose we are now entering number 5 into a calculator - meaning multiply whatever is on the display (s) by 10 and adding 5 to it (if nothing is on the display 0x10=0)

  $I\ [5\,] = s \Rightarrow 10s + 5$

in general

  $I\ [d\,] = s \Rightarrow 10s + val(d\,)$

- but if the system is modey a command may both change the state of the system and its mode, i.e. the meanings of the commands that follow

seeing through modes

- a mode information must also be taken into account ($m \in M$) for the interpretation of a command

  $I\ [c]\ m \in S \rightarrow (S \times M)$ or equivalently

  $I\ [c]\ m \rightarrow (S \times M) \rightarrow (S \times M)$

- example: a calculator may operate in "decimal" mode or in "hexadecimal" mode, the input digit does not alter the mode m (10 or 16)

  $I\ [d]\ m = s \Rightarrow (ms + val(d), m)$

insert or type?

- another example: a word processor may use *i* as a command for "insert" or *i* as a letter to be written in the text ... very confusing!

  I [*i*] m = s ⇒ (s '+ val(*i*), m+1)

- the *i* key has a meaning which depends on where it is inserted and it affects the meaning of the subsequent commands, the cursor position being represented by m
- if we have typed *i* what happens next is that any letter will be inserted to the right of the cursor position (m+1), s' would be the new state based on the old s but including the letter i at position m

solution

- the cursor position is always visible to the user, we could include it in the state S so as to make the word processor modeless, in this case we would have

  $I'[c] \in S' \rightarrow S'$ where $S' = S \times M$ this new state set conceals the mode M and now I' has a modeless form

bottlenecks locate modes

- to what extent can the mode be treated as a part of the system's state? the state can be factored in many ways...
- perhaps we may define the mode as a dependence on any info, data or procedure that is not apparent to the user (usability point of view)
- $I' [c] \in S' \rightarrow S'$ is modeless but if the component M of S' is concealed to the user then, the system is modey
- what to conceal and what to show this is Hamlet's dilemma in interface design

Some conclusions

- the quickest way to make something incomprehensible is to conceal something from the user, even better if what is concealed keeps changing: modes
- information not transferred between user & computer increases modiness
- WYSIWYG is a protypical guep
- the user is inspired to confidence (no hidden tricks)
- system model can be understood
- user may generalize his knowledge
- "out of sight out of mind": if something is out of sight make sure the user can operate with it out of mind